

Rozwiązywanie układów równań nieliniowych uproszczoną metodą Newtona

1 Zastosowanie

Procedura *SimplifiedNewton* rozwiązuje układ n równań nieliniowych uproszczoną metodą Newtona w zmien-nopozycyjnej arytmetyce klasycznej jak i przedziałowej postaci:

$$f_k(x_1, x_2, \dots, x_n) = 0, k = 1, 2, \dots, n$$

2 Opis metody

Układ równań nieliniowych (1) jest rozwiązywany uproszczoną wersją metody Newtona, w której kolejne przy-bliżenia rozwiązania są wyznaczane na podstawie następujących wzorów:

$$\begin{aligned} x_1^{(l+1)} &= x_1^{(l)} - \omega \frac{f_1(x_1^{(l)}, x_2^{(l)}, \dots, x_n^{(l)})}{\frac{\partial f_1}{\partial x_1}(x_1^{(l)}, x_2^{(l)}, \dots, x_n^{(l)})}, \\ x_2^{(l+1)} &= x_2^{(l)} - \omega \frac{f_2(x_1^{(l+1)}, x_2^{(l)}, \dots, x_n^{(l)})}{\frac{\partial f_2}{\partial x_2}(x_1^{(l+1)}, x_2^{(l)}, \dots, x_n^{(l)})}, \\ &\dots \\ x_n^{(l+1)} &= x_n^{(l)} - \omega \frac{f_n(x_1^{(l+1)}, \dots, x_{n-1}^{(l+1)}, x_n^{(l)})}{\frac{\partial f_n}{\partial x_n}(x_1^{(l+1)}, \dots, x_{n-1}^{(l+1)}, x_n^{(l)})} \end{aligned} \quad (1)$$

gdzie $x_k^{(0)}$ ($k = 1, 2, \dots, n$) oznacza początkowe przybliżenie niewiadomej x_k , a ω – parametr procesu itera-cyjnego, taki że $0 < \omega < 2$ (zwykle przyjmuje się, że $\omega = 1$).

W zależności od wybranego trybu pracy (*software_mode*), obliczenia prowadzone są w klasycznej arytmetyce zmiennoprzecinkowej lub w arytmetyce przedziałowej. Choć oba warianty realizują ogólny schemat iteracyjny Gaussa-Seidela opisany układem wzorów (2), to ich fizyczna realizacja algorytmiczna w kodzie źródłowym przebiega odmiennie:

- **W wariancie klasycznym (SimplifiedNewton_Point):** Wzór (2) jest implementowany wprost (*ver-batim*). Funkcje układu f_i oraz ich pochodne cząstkowe $\frac{\partial f_i}{\partial x_i}$ są ewaluowane bezpośrednio w punktowych koordynatach aktualnego wektora przybliżeń $x^{(l)}$.
- **W wariancie przedziałowym (SimplifiedNewton_Interval):** Ze względu na specyfikę arytmetyki interwałowej, ogólny wzór (2) poddawany jest modyfikacji stabilizującej zbieżność. Dla każdego kroku pętli wewnętrznej i , korekta Newtona wyznaczana jest według zmodyfikowanego, rygorystycznego schematu różnicowego:

$$[N_i] = m([x_i]) - \omega \frac{f_i([x_1^{(l+1)}], \dots, [x_{i-1}^{(l+1)}], \mathbf{m}([x_i]), [x_{i+1}^{(l)}], \dots, [x_n^{(l)}])}{\frac{\partial f_i}{\partial x_i}([x_1^{(l+1)}], \dots, [x_{i-1}^{(l+1)}], [\mathbf{x}_i], [x_{i+1}^{(l)}], \dots, [x_n^{(l)}])} \quad (2)$$

gdzie $m([x_i])$ oznacza punktowy środek przedziału wyznaczony sprzętowo przez procedurę *Int_Avg*. Nowe przybliżenie przedziałowe na koniec każdego kroku pętli wewnętrznej jest obowiązkowo obcinane za pomocą operacji przecięcia zbiorów realizowanej przez procedurę *Int_Intersect*:

$$[x_i]^{(l+1)} = [x_i]^{(l)} \cap [N_i] \quad (3)$$

2.1 Kryteria stopu i kontrola zbieżności

Proces iteracyjny w obu zaimplementowanych procedurach sterowany jest dynamicznie za pomocą kryterium stopu opartego na zadanej dokładności ε (*eps*). Warunek zakończenia obliczeń realizowany jest sprzętowo na poziomie rejestru flag procesora:

- **W wariancie klasycznym (SimplifiedNewton_Point):** Po zakończeniu pętli wewnętrznej, aktualna norma maksimum błędów przybliżeń porównywana jest z wartością `eps` za pomocą instrukcji `fcomi`. Norma ta wyznaczana jest jako największa bezwzględna zmiana składowej wektora w danej iteracji:

$$\max_{k=1\dots n} \left| x_k^{(l+1)} - x_k^{(l)} \right| < \varepsilon \quad (4)$$

Jeśli wyznaczony błąd spełnia powyższą relację ($\max_error < \varepsilon$), instrukcja skoku warunkowego `jb end_loop` natychmiast przerywa wykonywanie pętli głównej, zwracając sterowanie do środowiska C++.

- **W wariancie przedziałowym (SimplifiedNewton_Interval):** Kryterium zbieżności badane jest poprzez analizę najszerszego wyznaczonego przedziału w danej iteracji (`max_width`), odłożonego pod bezpiecznym adresem `[rbp-96]`. Instrukcja `jb @sukces_interval` przerywa proces w momencie, gdy spełniony zostanie warunek:

$$\max_{k=1\dots n} \left(\text{width}([x_k]^{(l+1)}) \right) < \varepsilon \quad (5)$$

Następuje wówczas natychmiastowe przejście do sekcji przepisywania wyników roboczych ze stosu do oryginalnej tablicy pamięci RAM docelowej.

W przypadku braku osiągnięcia zbieżności numerycznej (np. dla układów rozbieżnych), bezpiecznikiem zapobiegającym utknięciu programu w nieskończonej pętli jest m.in. licznik `mit`. Po wykonaniu maksymalnej zadeklarowanej liczby iteracji (np. 150), pętla zostaje przerwana awaryjnie.

3 Wywołanie procedury

SimplifiedNewton(*n*, *x*, *f*, *df*, *omega*, *mit*, *eps*)

4 Dane

n – liczba równań układu (równa liczbie niewiadomych),

x – wskaźnik na tablicę, której elementy $x[i]$ ($i = 0, 1, 2, \dots, n-1$) zawierają początkowe przybliżenia rozwiązania,

f – wskaźnik na tablicę, której elementy $f[i]$ ($i = 0, 1, 2, \dots, n-1$) są procedurami języka C/C++, które na wskaźniku *r* zapisują wynik $f_i(x_0, x_1, \dots, x_{n-1})$, gdzie $x_0, x_1, \dots, x_{n-1}$ są elementami tablicy, na którą wskaźnik jest podany w parametrze *x*.

df – wskaźnik na tablicę, której elementy $df[i]$ ($i = 0, 1, 2, \dots, n-1$) są procedurami języka C/C++, które na wskaźniku *r* zapisują wynik $df_i(x_0, x_1, \dots, x_{n-1})$, czyli pochodnej $\frac{\delta f_i}{\delta x_i}$, gdzie $x_0, x_1, \dots, x_{n-1}$ są elementami tablicy, na którą wskaźnik jest podany w parametrze *x*.

omega – wartość parametru procesu iteracyjnego (1) i (2).

mit – maksymalna dozwolona liczba iteracji

eps – bezwzględna dokładność rozwiązania (bezwzględna dokładność między dwoma kolejnymi przybliżeniami, albo szerokość przedziału w zależności od trybu, zob. 2.1)

Uwaga! Po wykonaniu procedury *SimplifiedNewton* wartości elementów tablicy *x* są zmienione.

5 Wyniki

x – zmodyfikowana tablica *x* z danych z przybliżonymi rozwiązaniami układu równań.

6 Inne parametry

Procedura nie przyjmuje innych parametrów, a do kontaktu z użytkownikiem używa rejestrów kontrolnych FPU X87. W przypadku napotkania problemów funkcja podaje błąd dzielenia przez zero, albo nieprawidłowej operacji. Wtedy pod wynikami są wartości nieprzewidywalne. Pozostałe błędy są skutkiem normalnej pracy tej i innych procedur i należy je ignorować. Każdorazowo przed wywołaniem tej funkcji należy wyczyścić status błędów fpu korzystając z polecenia *fnclcx*, albo procedury *ClearX87Errors*.

7 Typy parametrów

```
long long      :  $n, mit$ 
double_num*    :  $x, \omega$ 
extended_double* :  $\epsilon$ 
void(*f[])(double_num*, double_num*) :  $f, df$ 
```

8 Identyfikatory nielokalne

extended_double – struktura 10 bajtowa (80-bitowa) pozwalająca na ominięcie ograniczenia *long double* do 64 bitów przez kompilator MSVC.

double_num – union 20 bajtowy typów *extended_double* i *interval*, będącego strukturą dwóch *extended_double*.

Biblioteka *math_core* będąca niezbędna do działania tej procedury implementuje powyższe struktury i udostępnia interfejs pozwalający wykonywać działania wykorzystując pełnię możliwości architektury X87.

Wszystkie funkcje i procedury użyte w tej dokumentacji i programie mają implementację w tej lub standardowej dla języka C/C++ bibliotece.

9 Tekst procedury

```
1  SimplifiedNewton proc
2  sub rsp, 40          ; Shadow space dla x64 ABI
3
4  mov rax, software_mode ; Pobierz tryb (musi być EXTERN QWORD)
5  test rax, rax
6  jz @F               ; Skok do najbliższego @@ (Forward)
7
8  ; Tryb przedziałowy
9  call SimplifiedNewton_Interval
10 jmp ending
11
12 @@:
13 ; Tryb punktowy (float80)
14 call SimplifiedNewton_Point
15
16 ending:
17 add rsp, 40
18 ret
19 SimplifiedNewton endp
```

9.1 SimplifiedNewton_Point

```
1  SimplifiedNewton_Point proc
2  push rbx             ; Zabezpieczenie stałych rejestrów i zrobienie miejsca na stosie
3  push r12
4  push r13
5  push r14
6  push r15
7  push rsi
8  push rdi
9  push rbp
10 sub rsp, 120
11
12 mov rbx, rcx          ; Zachowanie parametrów w stałych rejestrach, n
13 mov r13, rdx          ; x
14 mov r14, r8           ; f
15 mov rbp, r9           ; df
16
17 mov r12, [rsp+120+8+8+8+8+8+8+32+8+8+48+8] ; mit - licznik funkcji, offsety zostały wyliczone na
18 ; podstawie powyższych operacji
19
20 main_loop:
21 fldz                 ; Ustawienie maksimum eps dla pojedynczej iteracji jako zero
22 push rbx             ; Zapisanie licznika n na stosie
23
24 mov rsi, r14          ; Pobranie adresów tablic i liczników do pętli iteracji Newtona
25 mov rdi, rbp
26 mov r15, r13
27
28 iter_loop:           ; Pętla wykonująca się dla wszystkich zmiennych
29
30 lea rcx, [rsp+48]
31 mov rdx, r13
32 mov rax, [rsi]
```

```

33     sub rsp, 32
34     call rax
35     add rsp, 32           ; Wywołanie f_i
36
37     add rsi, 8           ; Przesunięcie wskaźnika tabeli f
38
39     lea rcx, [rsp+72]
40     mov rdx, r13
41     mov rax, [rdi]
42     sub rsp, 32
43     call rax
44     add rsp, 32           ; Wywołanie df_i
45
46     add rdi, 8           ; Przesunięcie wskaźnika tabeli df
47
48     lea rcx, [rsp+48]
49     lea rdx, [rsp+72]
50     lea r8, [rsp+48]
51     call ED_Div           ; Dzielenie f_i/df_i
52
53     lea rcx, [rsp+48]
54     mov rdx, [rsp+120+8+8+8+8+8+8+32+8+8+48+8]; omega
55     lea r8, [rsp+48]
56     call ED_Mul           ; Przemnożenie przez omega
57
58
59     lea rcx, [r15]
60     lea rdx, [rsp+48]
61     lea r8, [r15]
62     call ED_Sub           ; Odjęcie poprawki
63
64     fld tbyte ptr [rsp+96+8] ; Obliczanie eps i porównywanie z maksimum
65     fld tbyte ptr [rsp+48]
66     fabs
67     fcomi st(0), st(1)
68     fcmovb st(0), st(1)
69     fstp tbyte ptr [rsp+96+8]
70     fstp st(0)
71
72     add r15, 20
73     dec rbx
74     jnz iter_loop
75
76     pop rbx
77     mov rdx, [rsp+120+8+8+8+8+8+8+8+32+8+8+48+8] ; eps
78     fld tbyte ptr [rdx]
79     fld tbyte ptr [rsp+96]
80     fcomi st(0), st(1)
81     fstp st(0)
82     fstp st(0)
83     jb end_loop ; Sprawdzenie warunku eps
84     dec r12
85     jnz main_loop ; Sprawdzenie warunku mit
86
87     end_loop: ; Przywrócenie stosu
88     add rsp, 120
89     pop rbp
90     pop rdi
91     pop rsi
92     pop r15
93     pop r14
94     pop r13
95     pop r12
96     pop rbx
97     ret
98
99     SimplifiedNewton_Point endp

```

9.2 SimplifiedNewton_Interval

```

1  SimplifiedNewton_Interval proc
2  push rbp
3  mov rbp, rsp
4  push rbx
5  push r12
6  push r13
7  push r14
8  push r15
9  push rsi
10 push rdi           ; Ustawienie stosu
11
12 mov rax, rcx
13 imul rax, 20
14 add rax, 512
15 add rax, 15
16 add rax, 32
17 and rax, -16
18 sub rsp, rax       ; Dynamiczne przesunięcie stosu dla tablicy pomocniczej

```

```

19
20 mov rbx, rcx
21 mov r13, rdx
22 mov r14, r8 ; Zapisanie parametrów do zmiennych
23
24 mov [rbp-64], rbx
25 mov [rbp-72], r9
26 mov [rbp-80], rdx ; Zapisanie ważnych wartości na stosie
27
28 mov rsi, r13
29 lea rdi, [rsp+32]
30 imul rcx, 20
31 rep movsb ; Kopiowanie tablicy x na tablicę pomocniczą
32
33 lea r13, [rsp+32] ; Zapisanie wskaźnika na tablicę pomocniczą w rejestrze
34
35 mov r12, [rbp + 104] ; Pobranie mit do rejestru r12
36
37
38 main_loop:
39 fldz
40 fstp tbyte ptr [rbp-96] ; Wstawienie zera dla maksymalnej szerokości przedziału w iteracji
41
42 mov rbx, [rbp-64] ; Wzięcie licznika n ze stosu
43
44 mov rsi, r14 ; rsi = wskaźnik na tablicę funkcji
45 mov rdi, [rbp-72] ; rdi = wskaźnik na tablicę pochodnych
46 mov r15, r13 ; r15 = wskaźnik na bieżący przedział w pętli
47
48 iter_loop:
49
50 lea rcx, [r15]
51 lea rdx, [rbp-128]
52 call Int_Avg
53
54 lea rcx, [rbp-128]
55 lea rdx, [rbp-128]
56 call Int_LoadNum ; Zrobienie przedziału punktowego z x_i
57
58 fld tbyte ptr [r15]
59 fstp tbyte ptr [rbp-224]
60 fld tbyte ptr [r15+10]
61 fstp tbyte ptr [rbp-214] ; Zapisujemy oryginalny przedział na stosie
62
63 fld tbyte ptr [rbp-128]
64 fstp tbyte ptr [r15]
65 fld tbyte ptr [rbp-118]
66 fstp tbyte ptr [r15+10] ; Podmieniamy przedział, przedziałem punktowym
67
68
69 lea rcx, [rbp-160]
70 mov rdx, r13
71 mov rax, [rsi]
72 call rax ; wywołanie f_i
73
74
75 add rsi, 8 ; Następny wskaźnik funkcji
76
77
78 fld tbyte ptr [rbp-224]
79 fstp tbyte ptr [r15]
80 fld tbyte ptr [rbp-214]
81 fstp tbyte ptr [r15+10] ; Przywrócenie pełnego przedziału do liczenia pochodnej
82
83 lea rcx, [rbp-192]
84 mov rdx, r13
85 mov rax, [rdi]
86 call rax ; Liczenie pochodnej
87
88 add rdi, 8 ; Następny wskaźnik pochodnej
89
90 lea rcx, [rbp-160] ; f
91 lea rdx, [rbp-192] ; df
92 lea r8, [rbp-160]
93 call Int_Div ; Dzielimy f_i przez df_i i zapisujemy wynik tam gdzie był wynik f_i
94
95 lea rcx, [rbp-160]
96 mov rdx, [rbp + 96] ; Pobranie wskaźnika 'omega' (offset 96)
97 lea r8, [rbp-160]
98 call Int_Mul ; Mnożenie przez omegę
99
100 lea rcx, [rbp-128]
101 lea rdx, [rbp-160]
102 lea r8, [rbp-160]
103 call Int_Sub ; Odjęcie poprawki
104
105
106 lea rcx, [r15] ; oryginalny, stary przedział
107 lea rdx, [rbp-160] ; nowo wyliczony przedział
108 lea r8, [r15] ; nadpisujemy wynik bezpośrednio w kopii
109 call Int_Intersect ; Część wspólna

```

```

110
111     fld tbyte ptr [r15 + 10]
112     fld tbyte ptr [r15]
113     fsubp st(1), st(0)
114     fabs                                     ; Obliczanie szerokości przedziału
115
116     fld tbyte ptr [rbp-96]
117     fcomi st(0), st(1)
118     fcmovb st(0), st(1)
119     fstp tbyte ptr [rbp-96]
120     fstp st(0)                             ; Zapisanie większej szerokości (dla maksimum iteracji)
121
122     add r15, 20                             ; Następny interwał (wielkość 20 bajtów)
123
124     dec rbx
125     jnz iter_loop
126
127     mov rdx, [rbp + 112]                   ; Pobranie wskaźnika eps
128     fld tbyte ptr [rdx]
129     fld tbyte ptr [rbp-96]
130     fcomi st(0), st(1)                   ; Porównanie szerokości do eps
131
132     jb @sukces_interval
133
134     fstp st(0)                             ; Jeśli brak zbieżności, czyścimy koprocesor
135     fstp st(0)
136
137     dec r12                               ; Zmniejsz licznik mit
138     jnz main_loop
139
140     jmp @przepisz_i_wyjdz
141
142 @sukces_interval:
143 @przepisz_i_wyjdz:
144     mov rdi, [rbp-80]
145     mov rsi, r13
146     mov rcx, [rbp-64]
147     imul rcx, 20
148
149     jrcxz @puste_wyjście
150     rep movsb                             ; Skopiowanie tablicy tymczasowej do tablicy x.
151
152 @puste_wyjście:
153
154 end_loop:
155     pop rdi
156     pop rsi
157     pop r15
158     pop r14
159     pop r13
160     pop r12
161     pop rbx
162     mov rsp, rbp                         ; Bezpieczne i całkowite zniszczenie ramki stosu
163     pop rbp
164     ret
165 SimplifiedNewton_Interval endp

```

10 Przykłady

a) Układ równań:

$$2x_1^2 + x_2 + x_3 = 0$$

$$x_1x_2 - x_3 + 2 = 0$$

$$x_1x_3 - x_2x_3 = 0$$

Definicje procedur f i df (w formacie eksportowym DLL):

```

1     const double_num c2 = {
2         0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x00, 0x40, // .low
3         0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x00, 0x40
4     };
5     const extended_double _4 = { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x01, 0x40 };
6
7     // f1(x,y,z) = 2*x^2 + y + z = 0
8     __declspec(dllexport) void f1(double_num* ret, double_num* tab) {
9         double_num* x = &tab[0], * y = &tab[1], * z = &tab[2];
10        double_num x2, cx2, sum1;
11
12        M_PowInt(x, &c2.inter.high, &x2);
13        M_Mul(&c2, &x2, &cx2); // 2 * x^2
14        M_Add(&cx2, y, &sum1); // 2 * x^2 + y
15        M_Add(&sum1, z, ret); // 2 * x^2 + y + z
16    }
17

```

```

18 // f2(x,y,z) = x*y - z + 2 = 0
19 __declspec(dllexport) void f2(double_num* ret, double_num* tab) {
20     double_num* x = &tab[0], * y = &tab[1], * z = &tab[2];
21     double_num xy, sub1;
22
23     M_Mul(x, y, &xy); // x * y
24     M_Sub(&xy, z, &sub1); // x * y - z
25     M_Add(&sub1, &c2, ret); // x * y - z + 2
26 }
27
28 // f3(x,y,z) = x*z - y*z = 0
29 __declspec(dllexport) void f3(double_num* ret, double_num* tab) {
30     double_num* x = &tab[0], * y = &tab[1], * z = &tab[2];
31     double_num xz, yz;
32
33     M_Mul(x, z, &xz); // x * z
34     M_Mul(y, z, &yz); // y * z
35     M_Sub(&xz, &yz, ret); // x * z - y * z
36 }
37
38 // dF1/dx = 4x
39 __declspec(dllexport) void df1(double_num* ret, double_num* tab) {
40     double_num c4; M_LoadNum(&c4, &c4);
41     M_Mul(&c4, &tab[0], ret);
42 }
43
44 // dF2/dy = x
45 __declspec(dllexport) void df2(double_num* ret, double_num* tab) {
46     extended_double temp;
47     M_Mid(&tab[0], &temp);
48     M_LoadNum(&temp, ret);
49 }
50
51 // dF3/dz = x - y
52 __declspec(dllexport) void df3(double_num* ret, double_num* tab) {
53     M_Sub(&tab[0], &tab[1], ret);
54 }

```

Dane:

$n = 3$, $\omega = 1,0$, $\epsilon = 1E - 16$, $mit = 150$, $x[0] = 1,0$, $x[1] = -1,0$, $x[2] = -1,0$

Tryb: klasyczny

Zapisy z logów:

```

Loaded values:
x[0]: 10.000000000000000000E-1
x[1]: -10.000000000000000000E-1
x[2]: -10.000000000000000000E-1
Running SimplifiedNewton in float80 mode. With variables: mit=150, eps
=9.999999999999999999E-16, omega=10.000000000000000000E-1.
Results:
x[0]: 1.00000000000000001074E+0
x[1]: -1.9999999999999999784E+0
x[2]: 0.0E+0

```

b) Dla danych z a)

Dane:

$n = 3$, $\omega = 0,01$, $\epsilon = 1E - 14$, $mit = 150$, $x[0] = [1,0; 1,0]$, $x[1] = [-2,0; -2,0]$, $x[2] = [0,0; 0,00001]$

Tryb: przedziałowy

Zapisy z logów:

```

Loaded values:
x[0]: 10.000000000000000000E-1, [9.999999999999999998E-1;10.000000000000000000E-1]
x[1]: -2.000000000000000000E+0, [-2.000000000000000000E+0;-2.000000000000000000E+0]
x[2]: 4.999999999999999998E-6, [-3.645199531882474039E-4951;9.99999999999999970E-6]
Running SimplifiedNewton in pure interval mode. With variables: mit=150, eps
=9.999999999999999999E-14, omega=9.999999999999999998E-3.
Results:
x[0]: 10.000000000000000000E-1, [9.999999999999999998E-1;10.000000000000000000E-1], width
=5.42E-20
x[1]: -2.000000000000000000E+0, [-2.000000000000000000E+0;-2.000000000000000000E+0], width
=2.16E-19
x[2]: 4.949999999999999998E-6, [4.949999999999999978E-6;4.949999999999999988E-6], width
=1.24E-24

```

c) Dla danych z a)

Dane:

$n = 3$, $\omega = 0,01$, $\epsilon = 1E-4$, $mit = 150$, $x[0] = 1,0$, $x[1] = -2,0$, $x[2] = 0,0$

Tryb: przedziałowy z rozszerzeniem wejść

Zapisy z logów:

```
Loaded values:
x[0]: 10.000000000000000000E-1, [9.99999999999999988E-1;1.000000000000000001E+0]
x[1]: -2.000000000000000000E+0, [-2.000000000000000000E+0;-1.9999999999999999E+0]
x[2]: 0.0E+0, [-3.645199531882474039E-4951;3.6451995318824740392E-4951]
Running SimplifiedNewton in interval float data mode. With variables: mit=150, eps
=9.99999999999999970E-5, omega=9.99999999999999988E-3.
Results:
x[0]: 10.000000000000000000E-1, [9.99999999999999988E-1;1.000000000000000001E+0], width
=1.62E-19
x[1]: -2.000000000000000000E+0, [-2.000000000000000000E+0;-1.9999999999999999E+0], width
=3.25E-19
x[2]: 0.0E+0, [-3.645199531882474039E-4951;3.6451995318824740392E-4951], width=7.29E-4951
```

d) Dla danych z a)

Dane:

$n = 3$, $\omega = 0,01$, $\epsilon = 1E-14$, $mit = 150$, $x[0] = [0,85;1,15]$, $x[1] = [-2,01;-1,99]$, $x[2] = [-0,00001;0,00001]$

Tryb: przedziałowy

Zapisy z logów:

```
Loaded values:
x[0]: 10.000000000000000000E-1, [8.49999999999999990E-1;1.14999999999999998E+0]
x[1]: -2.000000000000000000E+0, [-2.010000000000000000E+0;-1.9899999999999999E+0]
x[2]: -4.49999999999999999E-5, [-9.9999999999999998E-5;9.9999999999999970E-6]
Running SimplifiedNewton in pure interval mode. With variables: mit=150, eps
=9.99999999999999931E-14, omega=9.99999999999999988E-3.
There were some critical errors, or Newton was not convergent. Try with different data. Error
code: 0x26.
```